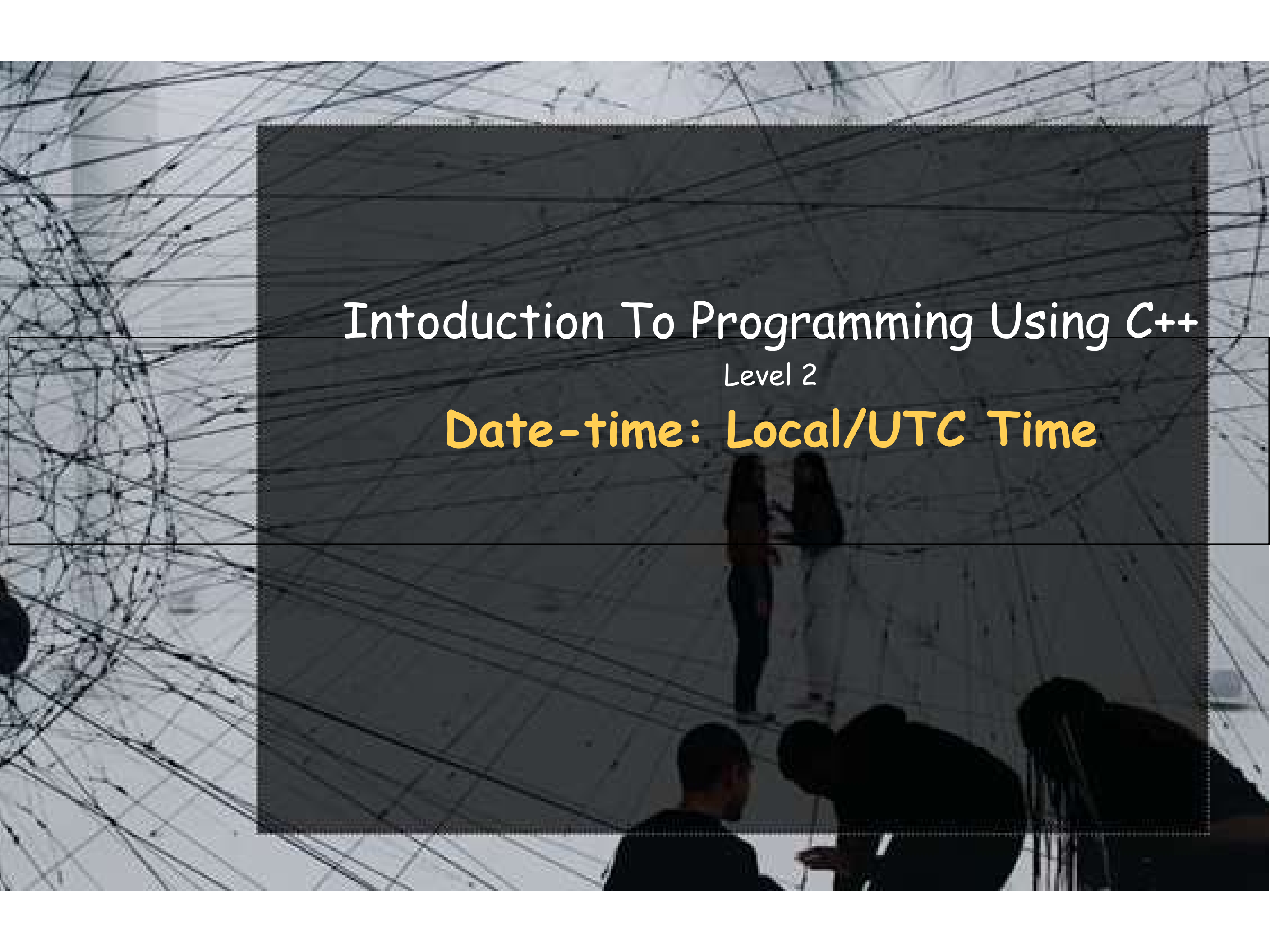




Introduction To Programming Using C++ Level 2

Date-time: Local/UTC Time



Introduction To Programming Using C++

Level 2

Date-time: Local/UTC Time

🔍 Definition

■ Getting the Current Time

▨ We use ``time()`` to get the current time.

▨ The returned value is stored in a ``time_t`` variable.

```
time_t t = time(0);
```

■ Local Date and Time

▨ We use ``ctime()`` to convert the current time into readable local date and time.

```
char* dt = ctime(&t);
```

■ UTC Date and Time

▨ We use ``gmtime()`` to convert the current time into a ``tm`` structure containing UTC time.

```
tm* gmtm = gmtime(&t);
```

■ Converting UTC to Text

▨ We use ``asctime()`` to convert the ``tm`` structure into readable text.

```
dt = asctime(gmtm);
```

Explanation

Saving Vector Data

 In the previous lesson, we loaded data from a file into a vector.

 Now we do the opposite: we take the data from the vector and write it back to a file.

Creating the Vector

 We first create a `vector<string>` containing the data we want to save.

```
vector<string> vFileContent{"Ali", "Shadi", "Maher", "Fadi", "Lama"};
```

Opening the File

 We open the file using `ios::out`.

 When an existing file is opened with `ios::out` alone, its previous contents are normally truncated before writing.

```
MyFile.open(FileName, ios::out);
```

Writing Vector Elements

 We use a range-based `for` loop to go through every element in the vector.

 `MyFile << Line << endl;` writes each element to the file on a separate line.

 The `const string&` form avoids copying each string because we only need to read it.

Closing the File

 After finishing the writing operation, we close the file.

```
MyFile.close();
```

Procedures

1. Include the `<ctime>` library.

2. If using Visual Studio and the older C-style time functions produce warning C4996, the warning can be disabled:

```
#pragma warning(disable : 4996)
```

3. Get the current time:

```
time_t t = time(0);
```

4. Convert the current time to local readable text:

```
char* dt = ctime(&t);
```

5. Display the local date and time:

```
cout << "Local date and time is: " << dt;
```

6. Convert the current time to UTC:

```
tm* gmtm = gmtime(&t);
```

7. Convert UTC time to readable text:

```
dt = asctime(gmtm);
```

8. Display the UTC date and time:

```
cout << "UTC date and time is: " << dt;
```

💡 Example

```
#include <iostream>
#include <ctime>
using namespace std;
int main()
{
    #pragma warning(disable : 4996)
    time_t t = time(0);
    char* dt = ctime(&t);
    cout << "Local date and time is: " << dt;
    tm* gmtm = gmtime(&t);
    dt = asctime(gmtm);
    cout << "UTC date and time is: " << dt;
    return 0;
}
```

➡ Result

Local date and time is: Thu Oct 09 15:45:20 2025

UTC date and time is: Thu Oct 09 13:45:20 2025

Note

- ◆ ``#pragma warning(disable : 4996)`` is mainly relevant to Visual Studio when using these older C-style functions.
- ◆ It is not needed in VS Code unless the compiler itself produces a relevant warning.
- ◆ ``time(0)`` is equivalent to ``time(nullptr)`` in modern C++.

⊘ Warning

- ◆ ``ctime()`, `gmtime()`, and `asctime()`` are older C-style functions.
- ◆ For modern C++, ``<chrono>`` provides more modern date/time facilities.
- ◆ The returned text from ``ctime()`` and ``asctime()`` typically contains a trailing newline.



Comparison

▣ `ctime()`

- ✓ Converts `time\_t` to readable local date and time.
- ✓ Returns text.
- ✓ Example: `Thu Oct 09 15:45:20 2025`

▤ `gmtime()`

- ✓ Converts `time\_t` to UTC time.
- ✓ Returns a pointer to a `tm` structure.
- ✓ The `tm` structure contains separate date/time components.

✦ Summary

- ✓ `<ctime>` provides functions for working with date and time.
- ✓ `time_t` represents a point in time.
- ✓ `time()` gets the current time.
- ✓ `ctime()` converts the time to readable local date and time.
- ✓ `gmtime()` converts the time to UTC inside a `tm` structure.
- ✓ `asctime()` converts a `tm` structure into readable text.

Thank you 😊